Datawhale

动手学大模型应用开发 · 第 9 节

综合实践二
练习反馈与跨仓库迁移

在资料导读基线上增加练习与反馈



主讲：陈思州

L09.01-S01

本节学习路线

Datawhale

综合项目第二阶段 · 围绕学习者尝试增加闭环

新增能力必须沿用第八节的来源、状态和证据边界。

01 继承基线

资料地图、检索、引用、路径与运行状态

02 增加练习

练习契约、尝试快照和可信执行

03 增加反馈

程序事实、模型诊断、提示与学习历史

04 迁移回归

新资料、新基线和有区分力的用例

第九节的增量价值来自学习闭环，而不是再生成一版资料摘要。

反馈闭环要保留五类对象

Datawhale

反馈对象 • 每一环都要能回到上一环

可用反馈要保留“为什么这样说”与“学习者接下来做了什么”。

01	**目标与契约**	学什么，怎样算完成
02	**学习者尝试**	代码、回答或操作快照
03	**执行证据**	用例结果、日志和差异
04	**反馈与行动**	差距、提示和下一步

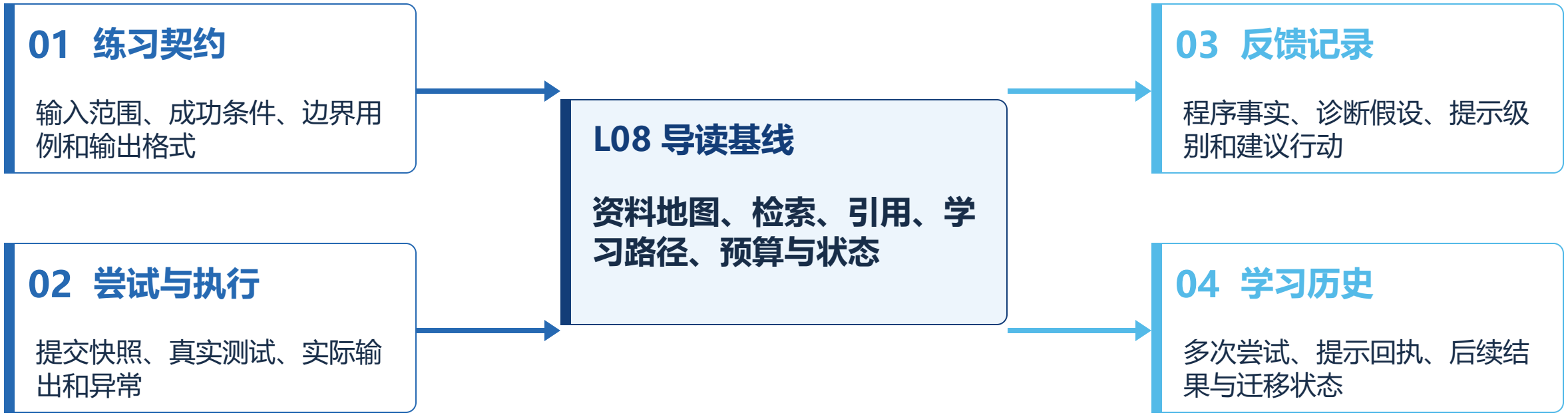
没有证据的反馈不可复查；没有后续行动的反馈也没有完成学习闭环。

第九节沿用资料底座，只增加学习闭环

Datawhale

增量边界 • 资料导读继续工作，学习闭环在其后追加

第二阶段只新增与尝试和反馈有关的对象，来源与证据规则保持不变。



第九节把“读懂资料”扩展为“尝试、得到反馈、修改并再次验证”。

导读结果进入练习反馈链

Datawhale

增量流程 • 每次反馈都对应一次固定尝试

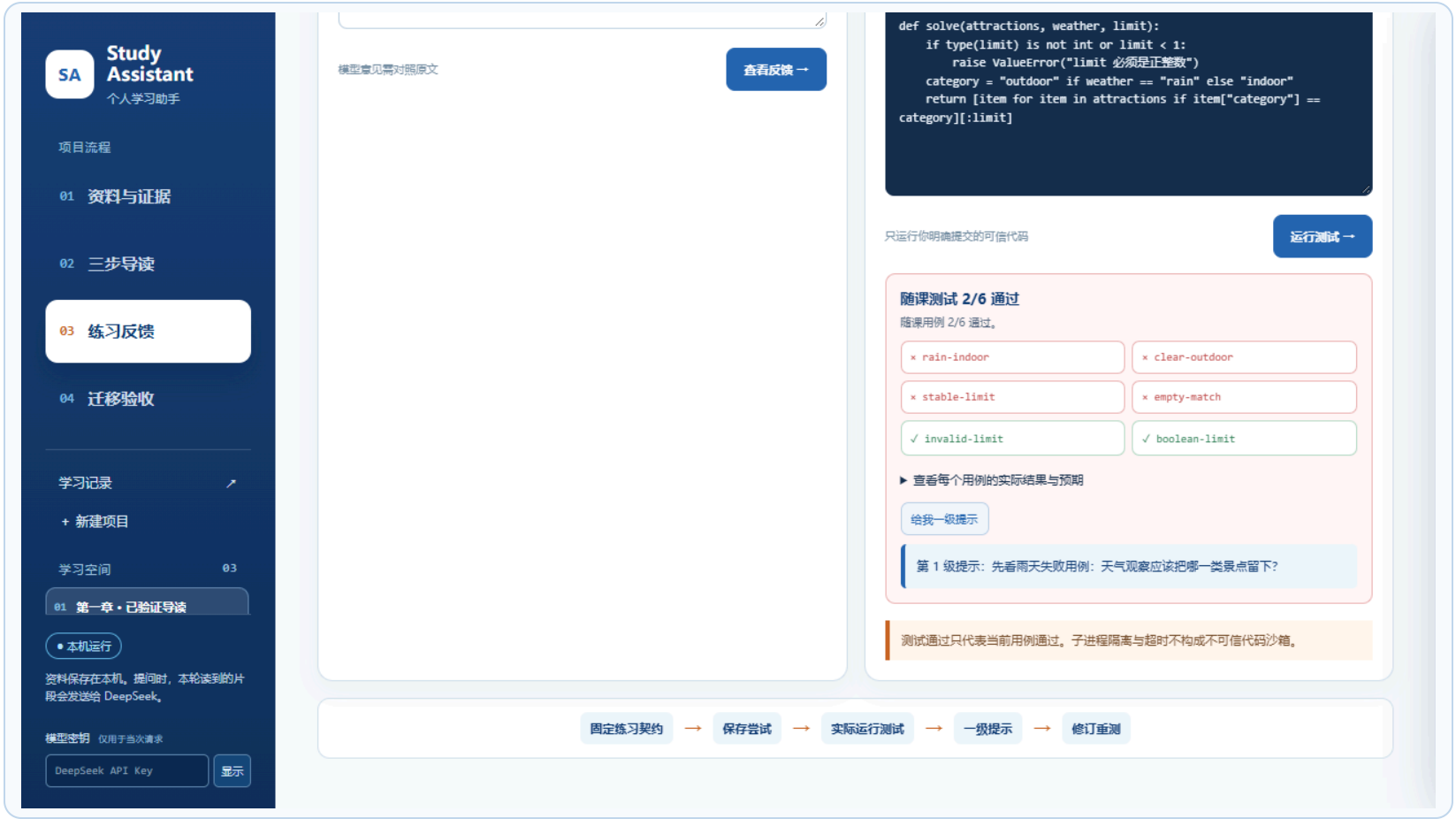
导读提供“学什么”，练习与反馈回答“实际做到哪里”。

01	学习路径	从资料证据中选择一个可练习目标
02	练习契约	定义输入、输出、正确性和边界用例
03	可信执行	运行提交，保存测试结果与实际输出
04	结构化反馈	先展示程序事实，再给诊断和最小提示
05	新尝试	修改后生成新快照并重新验收

系统状态只根据新执行证据更新，不根据模型说法更新。

项目画面：2/6 失败后给一级提示

Datawhale



实际网页画面 • 学生起始代码由 Python 子进程执行

先看程序事实，再看帮助

失败明确

起始提交实测 2/6，四个失败 ID 可逐项核查

一次一级

第一级只指出雨天输入，不交付完整代码

修订再验

Notebook 重测参考修订版 6/6，保留两次尝试

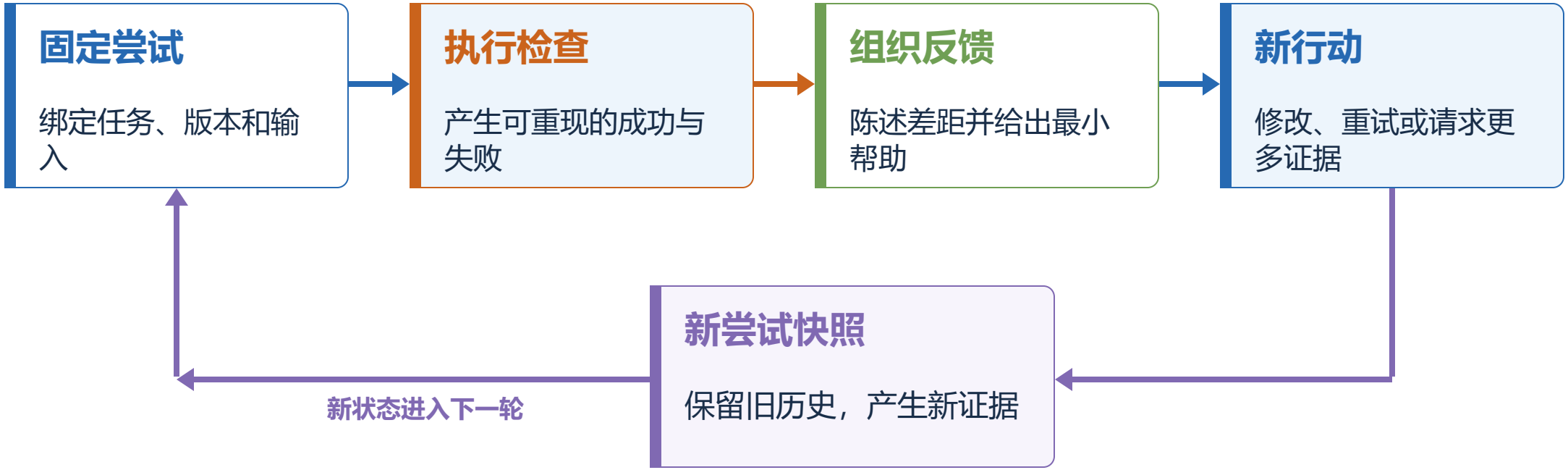
模型意见与提示不得覆盖程序产生的通过数和失败明细。

反馈、改进与迁移是同一条证据链

Datawhale

学习反馈闭环 · 新尝试必须产生新证据

反馈的价值不在于描述得多完整，而在于能否帮助学习者改变下一次行动。



学习者的新尝试是闭环中的独立对象，不能被反馈文本直接改写。

练习契约把学习目标变成可执行检查

Datawhale

练习契约 = 目标 + 输入范围 + 可观察结果 + 判定规则 + 失败语义。

它检查一次尝试是否满足已编码条件，不自动证明学习者已经掌握更广泛的知识。



打卡成功，用例通过?

程序正常退出，
3 个测试只通过了 2 个。

设计记录：完成状态和通过数量应该怎样保存?

尝试快照与运行状态回答不同问题

Datawhale

A

尝试快照

保存学习者提交的代码或回答、任务与资料版本、执行结果和时间。已记录后不覆盖。

边界

B

运行状态

保存 Agent 当前轮次、工具观察、预算和停止原因。它可恢复，但不能篡改历史尝试。

学习状态不能由一次尝试直接改写

Datawhale

A

尝试证据

记录某次提交、执行结果和反馈回执，事实固定且可以复查。

边界

B

学习状态

综合多次证据形成当前掌握度、常见障碍和下一目标，可更新但要保留依据。

有效反馈同时回答目标、差距与下一步

Datawhale

反馈三问 · 从成功条件走到可检查的新行动

一条反馈要让学习者知道标准、看到当前证据，并能完成下一步。

可行动反馈　　基于当前尝试，指向下一次可检查行动

01 要去哪里

成功条件和质量标准

02 现在在哪里

实测结果与标准的差距

03 下一步做什么

一个可完成、可重试的行动

04 完成后怎样检查

新尝试将产生什么新证据

反馈的终点不是文本生成完成，而是学习者能开始一次有意义的新尝试。

反馈时机取决于任务目标与错误成本

Datawhale

A

即时反馈

适合语法、格式和高频基础动作，迅速阻止错误累积，但可能打断完整思考。

边界

B

延迟反馈

适合综合任务和自我检查，保留探索空间，但必须防止错误概念持续固化。

提示应按必要程度逐层披露

Datawhale

渐进披露 • 只提供当下能推动思考的最小帮助

只有当前层无法推动下一次尝试时，才继续披露下一层。

实测证据

显示失败用例和实际输出

1

2

定位差距

指出需要观察的条件或步骤

3

拆解问题

把任务化成可回答的小问题

4

方法或解法

在必要时给更具体的支持

帮助的精度要与学习者当前证据匹配，而不是与模型能写多长匹配。

结构化反馈记录连接证据与下一次尝试

Datawhale

反馈对象 • 每项建议都能追溯到实际观察

只有一段自然语言时，系统无法区分事实、解释与建议，也无法安全重放。

可复查反馈记录　　attempt、criterion、evidence、level、action 和 receipt 共同关联

01 目标条件

本条反馈针对哪个成功标准或边界

02 观察证据

实际输入、输出、错误、测试和资料版本

03 诊断假设

可能原因及其证据范围，允许未知和修订

04 建议行动

下一次可以执行并检查的最小改动或问题

反馈记录把一次解释变成下一轮可验证的学习行动。

程序证据与模型解释承担不同责任

Datawhale

A

程序与规则

执行用例、记录输出、统计通过与失败、检查版本和边界。结果应可重现。

边界

B

模型解释

根据已读证据解释差距、提问或给当前层提示。不能改分、修改历史或自称掌握。

可信反馈要沿证据、诊断、提示与行动递进

Datawhale

反馈信任链 · 上层说法必须能回到下层证据

反馈可以继续丰富，但每一层都不能越权修改它所依赖的事实。

01	**实测证据**	尝试快照、用例结果和输出
02	**可复查诊断**	对应未满足的契约条件
03	**当前层提示**	不直接篡改成绩或学习状态
04	**新行动与回执**	学习者采取行动后生成新快照

事实、诊断、建议和行动分层保存，错误才能定位在正确的责任层。

模型诊断是可修订假设，不是学习事实

Datawhale

A

可确认事实

代码输出、测试状态、提交内容、资料版本和学习者明确表达的目标。

边界

B

诊断假设

模型推测的误解原因、思路缺口或下一步策略，需要后续提问和尝试验证。

学习历史与 Agent 运行需要分开恢复

Datawhale

双状态设计 · 一条学习历史可以包含多次独立 Agent 运行

学习历史回答“学习者怎样变化”，Agent 运行回答“这次反馈任务怎样执行”。

尝试 ID 绑定学习者、资料版本、练习与提交快照

01	学习历史	多次尝试、提示回执与进展
02	Agent 运行	轮次、工具、预算与停止状态
03	恢复规则	只恢复与当前快照匹配的任务
04	新提交	新快照、新反馈任务，旧历史保留

分开状态才能既恢复系统运行，又不改写学习者真实经历。

个性化反馈需要最小化学习者数据

Datawhale

数据治理 • 个性化程度不能超过证据和授权范围

保存更多历史会增加泄露、误判和旧状态长期影响新任务的风险。

最小学习记录 为当前目标保留必要证据，并明确谁能用、用多久

01	必要字段	练习、尝试、证据、提示级别和下一目标
02	保存期限	区分临时运行数据、课程记录和可删除推断
03	可见范围	按学习者、课程、角色和资料版本限制访问
04	更正与删除	允许纠正推断，删除不再需要的个性化状态

个性化以最少且可纠正的数据实现，不以完整监控换取方便。

提示级别只能在新回执中推进

Datawhale

幂等提示 • 恢复与重试不应改变教学进度

提示级别的变化必须来自新的学习者请求或新尝试，而不是基础设施重试。

1 逻辑请求

生成稳定 request_id

2 生成回执

保存当前提示级别与内容

3 重放同一请求

返回原回执，不推进级别

4 新请求或新尝试

根据新证据决定是否提升

幂等回执既保护工程可恢复性，也保护学习者不被意外越级提示。

项目画面：换资料先比较指纹

Datawhale



实际网页画面 • 两份不同资料的版本比较

迁移识别与后续验收

流程可复用

两份资料都走导入、检索、回查和练习流程

旧证据失效

资料指纹不同，第一章引用不能支持 Mini Guide 结论

目标需重验

新问题、引文和用例要在目标空间分别检查

网页的指纹对比是迁移入口，不等于目标资料已通过完整验收。

迁移矩阵区分保持项与重建项

Datawhale

迁移设计 • 对每个对象明确处理方式

只记录新仓库能够启动，无法证明目标、证据和反馈仍然有效。

迁移差异清单 对象、旧值、新值、处理方式和验收用例逐项对应

01 保持不变

快照结构、回执语义、证据分层和审计字段

02 参数适配

路径、命令、文件类型、解析器和资源限制

03 重新取证

入口、依赖、事实结论、坐标和支持关系

04 重新定义

学习目标、成功规则、边界用例和迁移基线

迁移报告要说明每一项为何保留、如何改变以及怎样验证。

回归用例必须能让修改前后产生不同结果

Datawhale

有区分力的回归用例要命中被修改的条件，并且在修改前失败、修改后通过。

如果改了 limit，但输入本来就不会超过 limit，输出不变不能证明该条件生效。

失败复盘要先定位责任层，再决定改什么

Datawhale

失败定位 • 反馈质量由多个责任层共同决定

不要默认“回答不好”就是模型问题；先查最靠近事实的层。

01	**契约与执行**	标准、版本、用例和实际输出
02	**诊断与提示**	失败条件、差距和帮助层级
03	**状态与恢复**	尝试快照、回执和任务恢复
04	**迁移与回归**	新契约、新基线和区分性用例

修改应落在失败所在层；不要用提示工程遮盖错误的契约、数据或状态。

同一个学习助手目标可以用不同实现方式

Datawhale

实现方式	需要拿出的关键证据
Notebook 原型	可运行代码、保存的结果、重启后的记忆与新资料复测
命令行应用	启动命令、工具轨迹、工作区文件、失败与迁移记录
网页应用	前后端源码、交互演示、状态保存与复现步骤
自研或现成框架	模块职责、调用记录、测试结果与能力边界

参考文献与延伸阅读

Datawhale

本节关于形成性反馈、提示层级、学习记录与 Agent 评估主要参考以下资料。

[1] Hattie, John, and Helen Timperley
The Power of Feedback
Review of Educational Research, 2007 · DOI:10.3102/003465430298487

[2] Shute, Valerie J.
Focus on Formative Feedback
Review of Educational Research, 2008 · DOI:10.3102/0034654307313795

[3] Anthropic
Demystifying Evals for AI Agents
anthropic.com/engineering/demystifying-evals-for-ai-agents

[4] ADL Initiative
Experience API Specification
github.com/adlnet/xAPI-Spec

[5] Anthropic
Effective Context Engineering for AI Agents
anthropic.com/engineering/effective-context-engineering-for-ai-agents